

Cryptographic Encryption Vault

README based on the supplied `vault.py` script

1. Project Overview

This project is a Python command-line cryptographic vault that recursively encrypts all files inside a directory and can later restore them. The supplied script uses AES-256-GCM for file encryption and `script` for deriving a key that securely wraps the randomly generated vault key.

2. Objective

Build a secure multi-file directory encryption tool, manage the encryption key safely, restore the original directory structure, and verify that restored files are unchanged.

3. Security Design Used in `vault.py`

- **AES-256-GCM**: encrypts each file and authenticates its contents.
- **Random vault key**: a fresh 32-byte key is generated for each encryption operation.
- **script**: derives a 32-byte wrapping key from the master passphrase using $n=16384$, $r=8$, $p=1$.
- **Encrypted key backup**: the vault key is encrypted with AES-GCM and saved as `vault_key_backup.json`.
- **Random nonce**: a fresh 12-byte nonce is used for the vault-key backup and each encrypted file.
- **Additional authenticated data**: the relative filename is authenticated so filename tampering is detected.
- **SHA-256 verification**: the script includes `verify_directories()` to compare original and restored files byte-for-byte.

4. Requirements

Python 3.10+ and the cryptography package are required.

```
python -m pip install cryptography
```

5. Project Structure

```
cryptographic_vault_project/  
■■■ vault.py  
■■■ README.md  
■■■ README.pdf  
■■■ screenshots/  
    ■■■ 01_encryption_execution.png  
    ■■■ 02_decryption_execution.png  
    ■■■ 03_sha256_verification.png
```

6. Encryption Execution

Run the exact supplied script with:

```
python vault.py encrypt "original" "vault"
```

The program asks for a master passphrase, recursively processes files, and prints the number of encrypted files and the secure key-backup path.

7. Decryption Execution

Run:

```
python vault.py decrypt "vault" "restored"
```

The same master passphrase is required. The encrypted files are decrypted and restored using the relative filenames stored in their authenticated headers.

8. Verification

The supplied code contains `verify_directories(original, restored)`, which compares the file lists and SHA-256 hashes. The execution proof below shows a successful verification with all three demo files matching.

9. Execution Screenshots

Encryption

vault — Encryption Execution

```
project/vault.py encrypt "/mnt/data/cryptographic_vault/  
GCM  
ographic_vault_project/demo_exact/vault/vault_key_backu
```

Decryption

vault — Decryption Execution

```
project/vault.py decrypt "/mnt/data/cryptographic_vault/  
GCM  
ographic_vault_project/demo_exact/vault/vault_key_backu
```

SHA-256 Verification

vault — File Verification

```
restored)
```

10. GitHub Submission Notes

Upload vault.py, README.md/README.pdf, and the execution screenshots to the GitHub repository. Do not upload real passwords, private keys, or confidential personal data. The passphrase used in the screenshots is only a demonstration passphrase.

11. Result

Encryption: 3 files encrypted successfully

Decryption: 3 files restored successfully

Verification: SHA-256 PASS; all files match byte-for-byte